Ruh.ai V1 Reference Pack Definition AI BuildOps Pack

Governed digital employees for intake, specification, implementation planning, build orchestration, evaluation, release, and post-release learning

This document defines the first horizontal reference pack for companies that build AI products. It instantiates the platform PRD and pack specification as a reusable operating pack that turns software delivery into a governed conveyor of digital employees instead of a chain of manual handoffs.

What this pack must accomplish

- Normalize messy inbound product, support, and incident signals into clean work items.
- Generate better specifications and plans before work is assigned to coding agents or engineers.
- Use digital employees to orchestrate build, eval, review, release, and post-release learning without removing human control from architecture or production.
- Stay horizontal across AI product companies by focusing on common SDLC workflows rather than one team's exact stack.

Document facts

Pack ID: ref.ai.buildops

Reference version: 1.0.0

Pack type: Function reference pack

Primary deployment model: One tenant trust boundary; one workspace per product or repo cluster

Conforms to: Ruh.ai V1 Employee Pack Specification

Outcome area	Problems removed	Primary workflows	Business measures
Work intake	Requests arrive through support threads, Slack, incidents, and product notes with poor deduplication or prioritization.	Signal intake and triage	Backlog hygiene, time-to-owner
Specification quality	Teams start coding from partial requirements and missing acceptance criteria.	Spec and planning	Spec completeness, rework rate
Implementation flow	Engineering context is scattered and coding agents are invoked without guardrails.	Plan -> build -> PR orchestration	Spec-to-PR time, review churn
AI quality gates	Prompt/model/tool changes reach users without enough regression coverage.	QA and eval gating	Eval pass rate, escaped regressions
Release and learning	Release notes, rollout checks, and incident feedback loops are manual	Release + incident learning	Release confidence, incident-to-RCA time

	and inconsistent.		

Why this pack stays horizontal

The pack does not assume a single repo host, issue tracker, or observability vendor. It standardizes the common operating spine of AI product delivery: intake, specification, planning, build orchestration, evaluation, release, and learning from production. Provider overlays bind those capabilities to each tenant's stack.

Executive snapshot

The AI BuildOps Pack is a governed digital-workforce bundle for teams that ship AI products. It accepts incoming signals from product feedback, support, incidents, and planning conversations; turns them into structured work; helps prepare technical plans; orchestrates coding-agent work where appropriate; runs tests and AI evals; packages release artifacts; and closes the loop by converting production failures into new regression cases.

The pack is intentionally not a replacement for GitHub, GitLab, Jira, Linear, CI/CD, or observability systems. Instead, it sits above them as an execution layer that assembles context, routes work to the right digital employee, enforces policy, and measures delivery performance.

Pack area	V1 reference definition
Employees	Intake & triage; spec; architecture planner; builder orchestrator; QA & evals; review & release; incident & feedback
Core workflows	Signal intake; spec generation; architecture plan; build orchestration; eval gating; release prep; incident learning
Supported capability groups	Issue tracking; repo/PR; docs; chat; CI/CD; observability; feature flags; model telemetry
Non-negotiable controls	No direct merge to protected branches; no production deploy without approval; AI changes require eval gates; full audit history
Pilot slice	Intake, spec, builder orchestration, QA/evals, and release on one product team

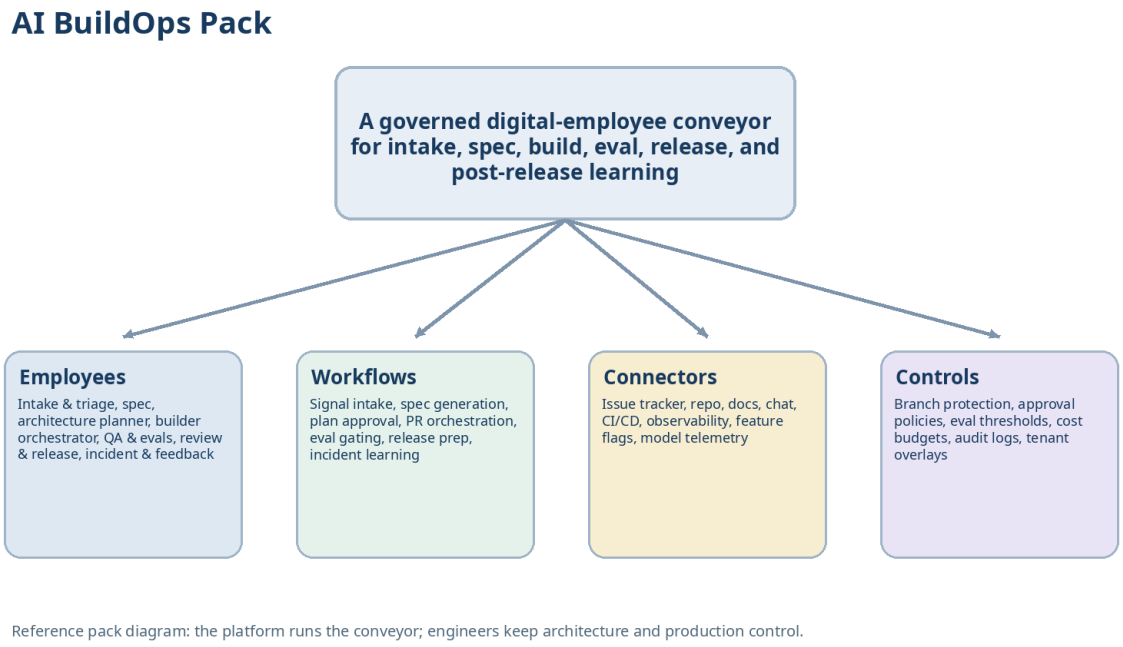

Figure 1. The AI BuildOps pack organizes digital employees around the common delivery conveyor used by AI product companies.

1. Purpose, scope, and design goals

The purpose of this pack is to make Ruh.ai useful beyond one industry by packaging a repeatable operating model for software teams, especially companies building AI-native products. The design assumption is that modern teams already have best-of-breed tools for code, tracking, documentation, CI, and observability, but lack one execution layer that can move work across those systems consistently.

The pack should reduce the coordination tax around software delivery without pretending that architecture judgment, code ownership, or production accountability can be fully delegated. Humans still decide what to build, what to merge, and what to ship; the digital employees remove repetitive assembly, analysis, and follow-through work around those decisions.

- Must work across common combinations of GitHub or GitLab, Jira or Linear, Slack or Teams, Notion or Confluence, and major CI/observability vendors.
- Must treat AI-specific quality gates as first-class: prompt changes, model-route changes, tool changes, and eval thresholds.
- Must keep protected-branch merges, deployment, and customer-facing incident communication behind explicit approvals.
- Must convert production failures and bad conversations into reusable regression cases.

1.1 Out of scope for V1

- Fully autonomous production deploys, automatic merges to protected branches, or unsupervised architectural changes.
- A replacement for the issue tracker, code host, docs system, or observability vendor.

- Optimization for lines of code written; the pack is judged by delivery quality and throughput, not code volume.

2. Target operating model and deployment assumptions

Dimension	Reference assumption	Why it matters
Tenant boundary	One company or one clearly isolated business unit per deployment boundary	Matches the platform trust and secret model.
Workspace model	One workspace per product or repo cluster; optional portfolio workspace for leadership analytics	Keeps context and approvals scoped to the right codebase.
Primary operators	Engineering manager, PM, tech lead, engineer, QA lead, SRE, support lead	These roles review and benefit from the artifacts.
System of record	At least one issue tracker, one code host, one docs system, and one CI pipeline are already in place	The pack writes back into real delivery systems.
AI feature surface	The team ships prompts, model routes, agent workflows, or other AI behaviors that can regress independently of code	Needed to justify eval-first workflow design.
Rollout mode	Pilot one team and one product area before multi-team rollout	Required to tune policies, stacks, and cost budgets.

Supported customer archetypes

- AI-native SaaS company shipping prompt, agent, or model-driven product features.
- B2B product team with strong code tooling but weak workflow orchestration across product, engineering, QA, and ops.
- Internal platform or applied-AI team that needs a governed delivery conveyor for multiple services.

3. Pack identity, dependency model, and exported assets

This is a function reference pack rather than a vendor-specific or industry-specific pack. It declares reusable workflow assets for product delivery and binds to provider adapters through overlay values.

Manifest field	Reference value
id	ref.ai.buildops
version	1.0.0
packType	function-reference

dependsOn	ruh.foundation.runtime; ruh.foundation.approvals; ruh.foundation.code; provider overlays
exports.employeeTemplates	7
exports.workflowTemplates	7
defaultPolicyProfile	ai-buildops-default
defaultEvalSuite	ai-buildops-core

3.1 Suggested source tree

```
ai.buildops/
  pack.yaml
  README.md
  employees/
    intake-triage.yaml
    spec-employee.yaml
    architecture-planner.yaml
    builder-orchestrator.yaml
    qa-evals.yaml
    review-release.yaml
    incident-feedback.yaml
  workflows/
    signal-intake.yaml
    spec-generation.yaml
    architecture-plan.yaml
    build-orchestration.yaml
    eval-gating.yaml
    release-prep.yaml
    incident-learning.yaml
  artifacts/
    work-item-brief.schema.json
    spec-package.schema.json
    technical-plan.schema.json
    eval-run-summary.schema.json
    release-brief.schema.json
    incident-rca.schema.json
  skills/
    repo-context/
    coding-agent-control/
    eval-ops/
    release-prep/
    incident-correlation/
  evals/
    triage/
    spec-quality/
    planner/
    policy/
    ai-regression/
  overlays/
    github-linear.overlay.yaml
    github-jira.overlay.yaml
    gitlab-jira.overlay.yaml
```

4. Canonical objects and delivery vocabulary

To stay horizontal, the pack normalizes the objects that matter across software delivery and AI feature iteration. Provider-specific IDs remain available in adapters, but workflow logic should use canonical objects.

Canonical object	Description	Representative source systems
Signal	Any raw inbound item such as support feedback, Slack thread, bug alert, or incident page.	Slack, email, support, observability
Work item	Normalized unit of work with owner, priority, linked evidence, and desired outcome.	Jira, Linear, internal graph
Spec package	Problem statement, user story, acceptance criteria, rollout notes, and risks.	Docs, issue tracker
Technical plan	Proposed implementation path including touched systems, files, migrations, and tests.	Docs, PR planning
Code change	Branch, commits, pull request, and linked CI runs.	GitHub, GitLab
Eval suite	AI and software checks that must pass for a change type.	Eval store, CI
Prompt/model route	Versioned AI behavior surface that may change independently of code.	Prompt registry, config repo
Release	Packaged change set, rollout plan, notes, and approvals.	CI/CD, release tooling
Incident	Production problem with customer or operational impact.	Sentry, Datadog, PagerDuty
Regression case	Test or eval derived from prior failure or bad conversation.	Eval store, test suite

Default execution flow for the AI BuildOps pack

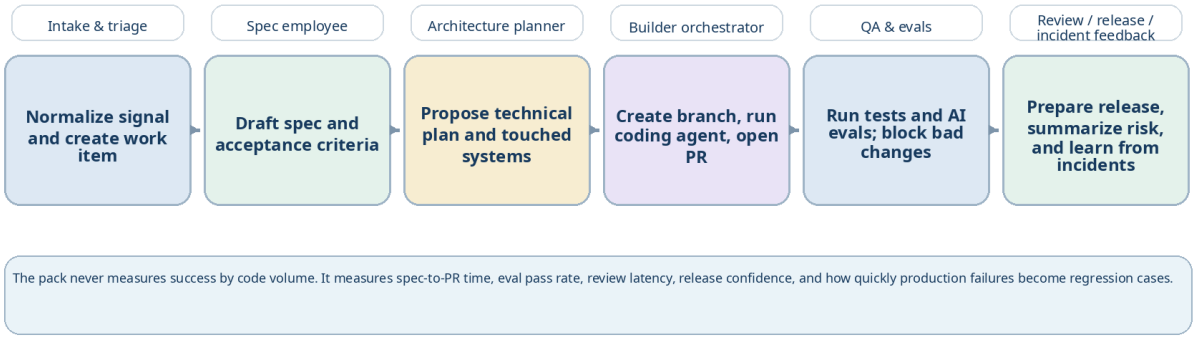

Figure 2. Default collaboration pattern for the AI BuildOps conveyor: a work item flows from intake through spec, planning, build, eval, release, and feedback.

5. Employee roster and collaboration contract

The pack exports seven default employees. Each employee has one primary responsibility and a narrow capability set. Collaboration happens through official workflow tasks and artifacts, never through hidden side channels.

Employee	Primary lane	Default risk tier	What success looks like
Intake & triage	Normalize inbound signal and create a clean work item	Low	Important signals are deduplicated, prioritized, and assigned fast.
Spec employee	Turn ambiguous requests into a usable spec package	Low	Teams start with clearer requirements and acceptance criteria.
Architecture planner	Propose implementation plan and touched systems	Low-Medium	Humans review better first-pass technical plans.
Builder orchestrator	Coordinate coding agents, branches, and PR creation	Medium	Changes arrive as reviewable PRs with context attached.
QA & evals	Run tests, generate regression cases, and block risky changes	Medium	AI and code regressions are caught before release.
Review & release	Prepare release artifacts, rollout notes, and enablement summaries	Medium	Release prep is faster and more consistent.
Incident & feedback	Correlate incidents, summarize RCA, and feed failures back into evals	Medium	Production issues create learning loops instead of one-off cleanup.

5.1 Intake & triage

Field	Definition
Purpose	Normalizes fragmented inputs across product, support, engineering, and operations into typed work items with evidence links and severity hints.
Primary triggers	Slack thread; support ticket; observability alert; product request; manually submitted work request.
Accepts tasks	signal.normalize; work-item.create; duplicate.cluster
Produces	Work-item brief; source bundle; severity recommendation; owner/team suggestion.

Capabilities	messaging.read; issue.read/write; docs.search; observability.read; task.write-internal
Default approval profile	Auto for internal work-item creation and routing. External responses remain human-controlled.
Key KPIs	Time-to-owner; duplicate rate; triage precision.

5.2 Spec employee

Field	Definition
Purpose	Turns messy requirements into a specification package that engineering and product can review before implementation starts.
Primary triggers	Approved work item; PM request; support escalation needing a product fix.
Accepts tasks	spec.generate; acceptance-criteria.expand; rollout-notes.prepare
Produces	Spec package; open questions; risk notes; rollout checklist draft.
Capabilities	docs.read/write-draft; issue.read/write-draft; messaging.draft
Default approval profile	Human review required before the spec becomes the official implementation target.
Key KPIs	Spec completeness; review acceptance; rework reduction.

5.3 Architecture planner

Field	Definition
Purpose	Produces a first-pass technical plan from the codebase, docs, and recent incidents so humans start from a stronger design baseline.
Primary triggers	Approved spec package; incident follow-up; change request touching multiple systems.
Accepts tasks	plan.generate; touched-systems.predict; migration-risk.assess
Produces	Technical plan; touched services/files; migration and dependency risks; test strategy draft.
Capabilities	repo.read; docs.read; search.index.read; issue.write-

	draft
Default approval profile	Plan approval required before a builder workflow starts on medium or high-risk changes.
Key KPIs	Plan acceptance; file-touch prediction quality; review churn reduction.

5.4 Builder orchestrator

Field	Definition
Purpose	Coordinates the mechanical execution steps around implementation by creating the right branch, invoking the right coding agent, opening the PR, and attaching context back to the work item.
Primary triggers	Approved technical plan; approved small change request; follow-up fix from eval failure.
Accepts tasks	build.start; branch.create; coding-agent.invoke; pr.open
Produces	Branch; pull request; linked CI run; implementation summary; follow-up tasks.
Capabilities	repo.read/write-limited; issue.update; ci.trigger; coding-agent.control
Default approval profile	No direct merge or protected-branch write. PR creation may be automated under policy.
Key KPIs	Spec-to-PR time; PR completeness; failure-to-retry handling.

5.5 QA & evals

Field	Definition
Purpose	Makes quality gates explicit for both classical software behavior and AI behavior by turning failures into tests and blocking risky changes before release.
Primary triggers	New PR; prompt/model/tool change; incident follow-up; bad production conversation.
Accepts tasks	test.run; eval.run; regression-case.generate; gate.decision
Produces	Eval run summary; regression bundle; blocked-

	change rationale; quality dashboard update.
Capabilities	ci.trigger; eval.store.read/write; repo.read; model-config.read
Default approval profile	Auto for eval execution. Any gate override requires human approval and is fully logged.
Key KPIs	Eval pass rate; escaped-regression reduction; regression-case growth.

5.6 Review & release

Field	Definition
Purpose	Packages approved changes into clear release artifacts, rollout notes, and internal enablement summaries.
Primary triggers	PR approved; release window reached; feature-flag change ready.
Accepts tasks	release.prepare; changelog.compose; rollout-checklist.generate
Produces	Release brief; changelog; customer/internal summary; rollout checklist.
Capabilities	repo.read; ci.read; docs.write-draft; messaging.draft; feature-flag.read
Default approval profile	Production deploy and external customer communication require explicit approval.
Key KPIs	Release prep time; change-summary quality; rollback readiness.

5.7 Incident & feedback

Field	Definition
Purpose	Closes the learning loop by correlating incidents, recent changes, and user feedback, then converting real failures into new tasks, tests, and evals.
Primary triggers	Incident created; postmortem request; repeated support complaint; eval miss in production.
Accepts tasks	incident.summarize; rca.draft; regression-case.backfill
Produces	Incident summary; RCA draft; linked code-change

	evidence; new regression cases; follow-up work items.
Capabilities	observability.read; repo.read; issue.write-draft; eval.store.write; docs.write-draft
Default approval profile	External incident communication stays human-owned. Internal RCA and follow-up drafts may auto-generate.
Key KPIs	Incident-to-RCA time; evidence-link quality; production-failure-to-regression conversion rate.

6. Workflow catalog

The reference pack organizes work as a conveyor. Each workflow is reusable pack content that can be composed into a tenant-specific operating model through overlays and policy settings.

Workflow	Primary trigger	Employee pattern	What the workflow does	Approval gate	Primary measures
Signal intake and triage	Incoming request, alert, or thread	Intake & triage	Cluster duplicates, link evidence, create work item, propose owner and priority	Usually auto	Time-to-owner; triage precision
Spec generation	Approved work item	Spec employee	Draft problem statement, acceptance criteria, rollout notes, and open questions	Spec review required	Spec acceptance; rework reduction
Architecture planning	Approved spec	Architecture planner	Propose touched systems, implementation path, migrations, and test plan	Plan approval for medium/high risk	Plan acceptance; review churn
Build orchestration	Approved technical plan	Builder orchestrator	Create branch, invoke coding agent, open PR, link CI and work item	No direct merge	Spec-to-PR time; PR completeness
Eval gating	New PR or AI behavior change	QA & evals	Run tests, generate regression cases, compare prompt/model variants, block bad changes	Gate overrides require approval	Eval pass rate; escaped regressions
Release	PR approved and	Review & release	Assemble release	Deploy approval	Release prep time;

preparation	ready to ship		brief, rollout checklist, summaries, and deployment metadata	required	rollback readiness
Incident learning	Incident or repeated failure	Incident & feedback	Draft RCA, link recent changes, create regression cases, open follow-up work items	External comms require approval	Incident-to-RCA time; learning-loop closure

6.1 MVP workflow slice

- Start with intake, spec generation, build orchestration, QA/evals, and release prep on one product team.
- Keep architecture planner in the MVP for non-trivial changes because it creates the context boundary for builder workflows.
- Bring incident & feedback online after the first shipping loop so real failures can generate regression cases.

7. Connector and capability profile

The pack binds to normal software-delivery capabilities rather than hardcoding one vendor stack. A tenant overlay chooses the actual providers and approval rules.

Capability group	Required?	Typical providers	Pack usage
issue.read/write	Yes	Jira; Linear	Create and update work items, specs, and follow-up tasks.
repo.read/write-limited	Yes	GitHub; GitLab	Read code context, create branches, and open PRs under policy.
docs.read/write-draft	Yes	Notion; Confluence; Google Docs; internal docs	Draft spec packages, plans, release notes, and RCAs.
messaging.read/draft	Yes	Slack; Teams; email	Capture inbound signals and prepare summaries or internal notifications.
ci.trigger/read	Yes	GitHub Actions; GitLab CI; Buildkite	Run tests, evals, and collect build status.
observability.read	Recommended	Sentry; Datadog; New Relic; PagerDuty	Enrich triage, RCA, and release decisions.
eval.store.read/write	Yes	Internal eval registry; vendor eval tooling	Store regression cases and gate rules.
feature-flag.read	Optional	LaunchDarkly; Statsig; internal config	Prepare rollout plans and blast-radius checks.
model-config.read	Recommended	Prompt registry; config repo;	Track prompt/model route

		gateway config	changes and required evals.

7.1 Minimum provider overlays

```
providers:
  issueTracker:
    adapter: linear | jira
  repo:
    adapter: github | gitlab
  docs:
    adapter: notion | confluence | gdocs
  messaging:
    teamChat: slack | teams
  ci:
    adapter: github-actions | gitlab-ci | buildkite
  observability:
    adapter: sentry | datadog | new-relic
policies:
  protectedBranches:
    - main
    - release/*
  requireHumanMerge: true
  requireHumanDeploy: true
  evalGateProfile: strict
```

8. Policy, approval, and safety model

The delivery conveyor becomes dangerous when automation can silently merge, deploy, or change protected configuration. The default policy profile therefore makes branch protection, eval gates, and deployment approvals explicit.

Representative action	Default treatment	Notes
Create internal work item or draft spec	Auto	Low-risk internal artifact creation.
Generate technical plan	Auto draft	Human review expected before medium/high-risk changes proceed.
Create branch, invoke coding agent, open PR	Auto under policy	Only for allowed repos/paths; no protected-branch writes.
Merge PR to protected branch	Approval required	Human-only in V1 unless explicitly overridden later.
Run tests and AI evals	Auto	Core control surface of the pack.
Override a failed eval gate	Approval required	Requires named approver and logged rationale.
Deploy to staging	Approval required or tenant policy	Allowed only where tenant enables it.
Deploy to production	Human only	Not automated in V1.
Send customer-facing incident update	Approval required	External communication remains human-owned.

Change secrets, credentials, or protected configs	Human only	Out of scope for unattended automation.

Approval rule design principle

The pack should automate assembly, comparison, and workflow progression early. It should not automate irreversible or high-blast-radius actions until the customer has operational confidence, eval maturity, and explicit approval design.

9. Canonical artifacts and sample contracts

All collaboration between employees and humans should pass through structured artifacts that can be stored, reviewed, versioned, and replayed. This is especially important for AI behavior changes, where the artifact is what makes the work auditable.

Artifact	Produced by	Required fields
Work-item brief	Intake & triage	signalRefs; classification; proposedOwner; severity; evidenceLinks; duplicateCluster
Spec package	Spec employee	problem; desiredOutcome; acceptanceCriteria; rolloutNotes; openQuestions
Technical plan	Architecture planner	touchedSystems; touchedFiles; migrationRisk; testStrategy; dependencies
Build summary	Builder orchestrator	branch; PRRef; codingAgentRunRefs; CIRefs; unresolvedItems
Eval run summary	QA & evals	changeType; suitesRun; passFail; criticalFindings; overrideRequired
Release brief	Review & release	changes; flags; rolloutPlan; rollbackPlan; approvals
Incident RCA	Incident & feedback	timeline; suspectedCause; evidence; relatedChanges; followUps
Regression case bundle	QA & evals / incident & feedback	testCases; evalCases; sourceFailureRefs; expectedBehavior

9.1 Sample workflow input

```
task:
  type: spec.generate
  workspaceRef: product:copilot-assistant
  sourceRefs:
    - slack:thread:4419
    - support:ticket:8931
```

```
    - issue:BUG-204
  requester:
    personRef: user:pm-17
  context:
    productArea: "conversation memory"
    severity: medium
    linkedIncident: INC-33
```

9.2 Sample eval run summary

```
artifact:
  type: eval-run-summary
  changeType: prompt-and-tooling
  suitesRun:
    - regression:conversation-memory
    - safety:prompt-injection
    - latency:assistant-core
  result: fail
  criticalFindings:
    - "Memory recall regression on 3 of 40 gold cases"
  overrideRequired: true
  followUpTasks:
    - issue:ENG-1172
```

10. Eval suite and release gates

A reference pack for AI product companies is only reusable if it can be validated repeatedly. The pack therefore ships eval families that test work-item quality, plan quality, AI regression coverage, policy enforcement, and release readiness.

Eval family	What it checks	Suggested release threshold
Triage evals	Correct work-item classification, deduplication, and routing	>= 0.90 weighted accuracy
Spec quality evals	Completeness of acceptance criteria, rollout notes, and open questions	>= 0.85 reviewer acceptance
Planner evals	Reasonable touched-system prediction and risk surfacing	>= 0.80 acceptance on reference set
Build-policy evals	No protected-branch writes, proper PR linkage, safe retries	100% pass on policy tests
AI regression evals	Prompt/model/tool behavior against gold sets and prior failures	No critical failures on required suites
Release readiness evals	Presence of rollback plan, approvals, and deployment metadata	100% on required checklist items
Incident learning evals	Quality of evidence linking and conversion to regression cases	>= 0.85 reviewer acceptance

- Any change that touches prompts, model routes, agent tools, or safety policies must map to an eval gate profile before release.
- Gate overrides require a named approver and a recorded reason.
- Every escaped production failure should create either a new test, a new eval case, or a policy rule update.

11. MVP deployment plan for the reference pack

Phase	What ships	Definition of success
Phase 1 - operating spine	Workspace setup, issue/repo/docs/chat connectors, intake & triage, spec employee	Requests arrive as clean work items with usable draft specs.
Phase 2 - execution loop	Architecture planner, builder orchestrator, PR linkage, CI hooks	Approved work regularly turns into reviewable PRs with context attached.
Phase 3 - quality gates	QA & evals, eval registry, regression generation, approval UX	Risky changes are blocked before release and failures create reusable evals.
Phase 4 - release and learning	Review & release, incident & feedback, dashboarding, overlay hardening	Release prep shrinks and production failures feed the conveyor.

Pilot success metrics

- Reduction in time from signal arrival to an owned, reviewable work item.
- Reduction in spec-to-PR cycle time for approved work.
- Increase in eval coverage for AI behavior changes.
- Reduction in escaped regressions and faster incident-to-RCA turnaround.

12. Example manifest and overlay excerpts

12.1 Manifest excerpt

```
id: ref.ai.buildops
version: 1.0.0
packType: function-reference
dependsOn:
  - ruh.foundation.runtime@^1.0
  - ruh.foundation.approvals@^1.0
  - ruh.foundation.code@^1.0
exports:
  employeeTemplates:
    - buildops.intake-triage.v1
    - buildops.spec-employee.v1
    - buildops.architecture-planner.v1
    - buildops.builder-orchestrator.v1
    - buildops.qa-evals.v1
    - buildops.review-release.v1
```

```
    - buildops.incident-feedback.v1
  workflowTemplates:
    - buildops.signal-intake.v1
    - buildops.spec-generation.v1
    - buildops.architecture-plan.v1
    - buildops.build-orchestration.v1
    - buildops.eval-gating.v1
    - buildops.release-prep.v1
    - buildops.incident-learning.v1
defaultPolicyProfile: ai-buildops-default
defaultEvalSuite: ai-buildops-core
```

12.2 Tenant overlay excerpt

```
tenant:
  name: "Sample AI Product Co"
  workspaceMode: per-product
providers:
  issueTracker:
    adapter: linear
  repo:
    adapter: github
  docs:
    adapter: notion
  messaging:
    teamChat: slack
  ci:
    adapter: github-actions
  observability:
    adapter: sentry
policies:
  protectedBranches:
    - main
    - release/*
  requireHumanMerge: true
  requireHumanDeploy: true
  evalGateProfile: strict
  codingAgentAllowedPaths:
    - services/assistant
    - prompts/
```

Appendix A. Runtime mapping example

Exported employee	Compiled OpenClaw agent id	Workspace bootstrap files	Primary skill groups
buildops.intake-triage.v1	prod-<id>-triage	SOUL.md; AGENTS.md; intake-playbook.md	signal-triage; issue-routing
buildops.spec-employee.v1	prod-<id>-spec	SOUL.md; AGENTS.md; spec-playbook.md	spec-authoring; rollout-notes
buildops.architecture-planner.v1	prod-<id>-planner	SOUL.md; AGENTS.md; planning-playbook.md	repo-context; plan-generation

buildops.builder-orchestrator.v1	prod-<id>-builder	SOUL.md; AGENTS.md; build-playbook.md	coding-agent-control; pr-orchestration
buildops.qa-evals.v1	prod-<id>-evals	SOUL.md; AGENTS.md; eval-playbook.md	eval-ops; regression-generation
buildops.review-release.v1	prod-<id>-release	SOUL.md; AGENTS.md; release-playbook.md	release-prep; changelog
buildops.incident-feedback.v1	prod-<id>-incident	SOUL.md; AGENTS.md; incident-playbook.md	incident-correlation; rca-drafting

Appendix B. Source assumptions

The references below anchor the OpenClaw runtime assumptions used by this reference pack and should be revisited whenever the runtime adapter changes.

[R1] OpenClaw CLI - agents (isolated agents with workspaces, auth, and routing) - https://docs.openclaw.ai/cli/agents

[R2] OpenClaw tools - skills (AgentSkills-compatible skill folders) - https://docs.openclaw.ai/tools/skills

[R3] OpenClaw automation - hooks - https://docs.openclaw.ai/automation/hooks

[R4] OpenClaw automation - cron jobs - https://docs.openclaw.ai/automation/cron-jobs

[R5] OpenClaw gateway - sandboxing - https://docs.openclaw.ai/gateway/sandboxing

[R6] OpenClaw gateway - security - https://docs.openclaw.ai/gateway/security